

Cognition-Oriented Design: A Paradigm for Intelligent Systems

A Comprehensive Framework for Human-Machine Collaboration

J. Francisco Bianchi Labriola

Independent author

August 24, 2025

Abstract

This paper introduces **Cognition-Oriented Design (COD)**, a novel design paradigm for intelligent systems that establishes structured practices for processes, classification, and communication. The paradigm consists of three core layers: a design philosophy for intelligent systems, a methodology for transforming events into structured execution, and a classification system for intelligence levels across flows and modules. We present the **Event-Derived Execution (EDE)** methodology, which processes events through derivation into execution units, and the **Machine Involvement Framework (MIF)**, which provides a logical system for classifying human and machine roles in task execution and process flows. The framework addresses the growing need for principled approaches to human-machine collaboration, offering a systematic way to design systems where machines can eventually lead while maintaining human oversight and control. Through detailed classification schemes and implementation guidelines, **Cognition-Oriented Design (COD)** provides a foundation for building scalable, intelligent systems that can evolve from human-led to machine-led execution as technology capabilities advance.

Keywords: Cognition-Oriented Design, Intelligent Systems, Human-Machine Interaction, Event-Driven Architecture, Machine Learning, Automation, Design Paradigms

Contents

1	Introduction	3
2	Overview	3
3	Machine Involvement Framework (MIF)	3
3.1	Classification Layers	4
3.2	Execution-Level Classification (ELC)	4
3.3	Flow-Level Classification (FLC)	4
3.4	Machine Involvement Definition	4
3.5	Usage Guidelines	5
3.5.1	Execution-Level Classification (ELC)	5
3.5.2	Flow-Level Classification (FLC)	5
4	Implementation Guidelines	5
4.1	ELC Availability Logic	5
4.2	Design Philosophy: E3-First	6
5	Event-Derived Execution (EDE)	6
5.1	Event	6
5.2	Derivation	7
5.3	Execution	7
5.4	Materials of EDE	7
5.5	Communication Standard	7
6	COD Structure	8
7	Further Dimensions	8
7.1	Components Perspective	8
8	Conclusion	9
9	Future Work	9

1 Introduction

The rapid advancement of artificial intelligence and machine learning technologies has created an urgent need for systematic approaches to designing intelligent systems that can effectively collaborate with humans. Traditional software design methodologies often treat machine intelligence as an afterthought, leading to systems that are either overly complex or insufficiently intelligent. **Cognition-Oriented Design (COD)** emerges as a response to this challenge, providing a principled framework for building intelligent systems through structured design practices.

Cognition-Oriented Design (COD) establishes a holistic approach that integrates three fundamental aspects: the design philosophy for intelligent systems, the methodology for processes, and the classification system for intelligence levels. This paradigm shift enables designers to think systematically about how human and machine capabilities can be orchestrated to achieve optimal outcomes.

The framework is built on the recognition that intelligent systems require more than just technical implementation—they need a coherent design philosophy that guides every aspect of system architecture, from the initial concept to the final implementation and ongoing evolution.

2 Overview

Cognition-Oriented Design (COD) is a paradigm for building intelligent systems through principled design practices that address the fundamental challenges of human-machine collaboration. The framework consists of three interconnected layers that work together to create a comprehensive approach to intelligent system design:

- **COD** establishes the design philosophy for intelligent systems, providing the foundational principles and values that guide system design decisions.
- **Event-Derived Execution (EDE)** defines the methodology for processes, transforming events into structured execution through a systematic approach to context management and action orchestration.
- **Machine Involvement Framework (MIF)** provides a classification system for intelligence levels across flows and modules, enabling precise understanding of how human and machine roles are distributed.

Together, these layers form a holistic approach: COD as the paradigm, EDE as the process engine, and MIF as the classification schema. This integration ensures that every aspect of system design is guided by consistent principles while maintaining the flexibility needed for real-world implementation.

3 Machine Involvement Framework (MIF)

The **Machine Involvement Framework (MIF)** represents a logical system for classifying how machine and human roles are distributed across tasks (Execution) and processes

(Flows). This framework provides the vocabulary and structure needed to precisely describe and analyze the intelligence characteristics of any system or process.

3.1 Classification Layers

The MIF operates at two distinct but related levels:

Table 1: Machine Involvement Framework Classification Layers

Layer	Group Name	Code Format	Purpose
Task	Execution-Level Classification (ELC)	E0 / E1 / E2 / E3	Define the role of human/machine
Flow	Flow-Level Classification (FLC)	F-A / F-B / F-C	Define system-wide patterns of

3.2 Execution-Level Classification (ELC)

The ELC defines how individual execution units are carried out—essentially answering the question "How is this specific task performed?" This classification operates at the granular level of individual actions within a system.

Table 2: Execution-Level Classification Details

Code	Label	Description
E0	Manual	Human performs the task without any machine involvement. No automation.
E1	Human-Initiated	Human performs the task and can request assistance (e.g., via button, prompt).
E2	Machine-Initiated	Machine initiates or completes the task; the environment is editable by human.
E3	Machine	Machine triggers, initiates and transitions the execution.

3.3 Flow-Level Classification (FLC)

The FLC describes how entire sequences of execution units operate, providing a system-level view of execution patterns. This classification helps understand the overall behavior and intelligence characteristics of complete workflows.

Table 3: Flow-Level Classification Details

Code	Label	Definition
F-A	Human-Only Flow	Every task is E0 — no machine interaction in the system.
F-B	Hybrid Flow	At least one task is E1 or E2 — human and machine both play roles.
F-C	Machine-Led Flow	The entire flow runs through machine triggers and automation. No human interaction.

3.4 Machine Involvement Definition

An execution unit or flow includes **machine involvement** if any **operation** (i.e., interaction with the environment) or **transition** (i.e., movement between steps) involves an input/output exchange with a machine. This definition ensures that even minimal machine interaction is properly classified and tracked.

3.5 Usage Guidelines

3.5.1 Execution-Level Classification (ELC)

ELC defines how an individual execution unit is carried out. It acts as a choice point—a "How do you want to do this?" selector—determining how the environment loads and operates. Not all ELC modes may be available at all times, depending on system capabilities and user permissions.

- **E0** – A basic environment is loaded with no machine assistance. Execution is entirely manual.
- **E1** – The environment remains similar to E0, but with tools available to optionally interact with machines (e.g., summarize, generate, check).
- **E2** – The environment loads with pre-filled or machine-generated content. Execution units are either partially or fully completed by the machine, with machine-powered tools enabled by default.

3.5.2 Flow-Level Classification (FLC)

FLC is informational and serves multiple purposes:

- How each step in a flow was executed (for completed flows)
- How machine support is available for the flow and its individual stages

FLC also guides system designers and builders toward machine-led optimization, helping teams visualize progress toward automation and identify where human-machine collaboration still plays a critical role.

4 Implementation Guidelines

The **Machine Involvement Framework (MIF)** is designed to ensure that Execution-Level Classifications (ELCs) are surfaced only when machine involvement provides meaningful value. Each ELC (E1–E3) must meet internal performance thresholds (speed, quality, or efficiency) before becoming available to users. Otherwise, the system defaults to lower ELCs (E0 or E1), or flags experimental modes as Beta.

4.1 ELC Availability Logic

- **E0** – Always available as a baseline
- **E1** – Available when machine assistance meaningfully supports human-led execution
- **E2** – Available when machine-initiated execution nears or outperforms E1/E0

- **E3** – Only exposed when machine-triggered, self-contained execution is reliable end-to-end

Each ELC is evaluated in the background via testing, logging, and user data—ELCs that do not meet the required threshold are withheld unless new, then shown with a Beta label.

4.2 Design Philosophy: E3-First

When building execution units, design with E2/3 in mind from the start—even if the current technology isn't "good enough" yet.

The goal is to architect systems and environments that are machine-executable by default, then work down to human-led fallbacks ($E2 \rightarrow E1 \rightarrow E0$) as needed.

This forward-first design approach:

- Avoids the trap of simply layering machine tools onto traditional software
- Ensures environments are structurally prepared for high automation
- Allows seamless upgrade paths as models, APIs, and cost structures improve
- Future-proofs flows—even if machine capabilities are temporarily withheld

By implementing this philosophy, designers are not just enabling machine assistance—they are building systems where machines can eventually lead.

5 Event-Derived Execution (EDE)

EDE is the process methodology of **Cognition-Oriented Design (COD)**, where systems are designed around the transformation of **Events** into **Execution Units (EUs)**. This methodology provides a systematic approach to understanding how external stimuli and internal intentions drive system behavior.

5.1 Event

The event represents the trigger point of the system, serving as the catalyst for all subsequent processing and execution.

- Can be **exogenous** (external, e.g., "a meeting occurred") or **endogenous** (internal, e.g., "user pressed start flow")
- Events represent both **facts** and **intentions**, providing the raw inputs that drive derivation
- Events can be simple (single occurrence) or complex (patterns of occurrences over time)

5.2 Derivation

Derivation represents the interpretation and shaping of events into actionable context. This stage is crucial for ensuring that the system's response to events is appropriate and effective.

- Operates through **Context Packages (CPs)**: structured artifacts that carry contextual information across the loop
- CPs standardize how context is created, stored, and pipelined into execution
- This stage ensures fidelity and continuity of meaning from event to action
- Derivation may involve multiple stages of processing and refinement

5.3 Execution

Execution represents the performance of an action within a defined environment. This is where the system's capabilities are applied to achieve the desired outcomes.

- Powered by **Execution Units (EUs)**: modular units that range from simple (e.g., composing an email) to complex (e.g., building a slide deck)
- EUs operate with context available to them, but are not required to consume or exhaust it
- All execution occurs within **Execution Environments (EEs)**: the "where" of action (applications, APIs, or machines)
- Execution may involve multiple EUs working in sequence or parallel

5.4 Materials of EDE

The EDE methodology relies on two key materials that enable its operation:

- **Context Packages (CPs)**: structured carriers of meaning and state that ensure information fidelity across the event-to-execution pipeline
- **Execution Environments (EEs)**: the domains in which EUs operate, providing the necessary infrastructure and capabilities

5.5 Communication Standard

Cognition-Oriented Design (COD) leverages **Model Context Protocol (MCP)** for standardized interaction with large language models and machines.

- MCP provides a uniform structure for LLM requests and responses

- CPs feed into MCP calls, ensuring consistency and interoperability
- This standardization enables seamless integration across different AI systems and platforms

6 COD Structure

The **Cognition-Oriented Design (COD)** framework is organized into a hierarchical structure that ensures comprehensive coverage of all aspects of intelligent system design:

- **Paradigm:** COD → establishes design philosophy and guiding principles
- **Process:** EDE → defines how events become executions through systematic methodology
- **Classification:** MIF → grades flows and modules by intelligence level and human-machine involvement
- **Materials & Communication:** CPs, EEs, and MCP → ensure interoperability and context fidelity across the system

This structure provides a systematic approach to designing intelligent systems, ensuring that no critical aspect is overlooked while maintaining the flexibility needed for real-world implementation.

7 Further Dimensions

Beyond the core layers of **Cognition-Oriented Design (COD)**(paradigm, process, classification), additional perspectives help extend its practical usage. These dimensions capture how **Cognition-Oriented Design (COD)** can be applied, adapted, and evolved in real systems.

7.1 Components Perspective

From a component perspective, **Cognition-Oriented Design (COD)** and EDE processes can be seen as modular building blocks, much like objects in object-oriented programming or components in React. Each Execution Unit (EU) can itself be designed as a self-contained component with its own flow logic and machine integrations.

This perspective generalizes machine integration into reusable units. For example, a text box with "improve writing" functionality built in becomes more than just a UI element—it is a machine-integrated component. As a flow, it may classify as E3 (fully machine-led within that micro-context), even while existing inside a broader E2 or E1 flow like a meeting log editor.

This creates reproducibility and scalability: once a machine-integrated component is designed, it can be reused across multiple environments and flows. A "smart text box"

might first appear in a meeting log builder, then resurface in a presentation creator or CRM notes tool, always carrying its embedded flow classification.

Viewing **Cognition-Oriented Design (COD)** through this component lens emphasizes cohesion and reusability. Intelligent systems stop being monolithic builds and instead emerge as libraries of interoperable, machine-integrated components, accelerating both design and evolution.

8 Conclusion

Cognition-Oriented Design (COD) represents a significant advancement in the field of intelligent system design, providing a comprehensive framework that addresses the fundamental challenges of human-machine collaboration. Through its three-layer architecture—paradigm, process, and classification—**Cognition-Oriented Design (COD)** offers a systematic approach to building systems that can evolve from human-led to machine-led execution as technology capabilities advance.

The framework's emphasis on E3-first design philosophy ensures that systems are built with future automation in mind, while the MIF provides the vocabulary and structure needed to precisely understand and optimize human-machine interaction patterns. The EDE methodology offers a systematic approach to transforming events into execution, ensuring that system responses are appropriate and effective.

As artificial intelligence and machine learning technologies continue to advance, the need for principled approaches to intelligent system design will only grow. **Cognition-Oriented Design (COD)** provides a foundation that can guide this evolution, ensuring that the benefits of machine intelligence are realized while maintaining human oversight and control.

The framework's component-based approach to machine integration enables the creation of reusable, scalable intelligent systems that can accelerate both design and evolution. By viewing intelligent systems as collections of interoperable, machine-integrated components, designers can build more effectively and efficiently.

Cognition-Oriented Design (COD) represents not just a technical framework, but a fundamental shift in how we think about and design intelligent systems. It provides the structure and methodology needed to build systems that can truly collaborate with humans, enhancing human capabilities while maintaining human agency and control.

9 Future Work

Several areas for future research and development have been identified:

- Development of automated tools for analyzing and classifying existing systems according to the MIF framework
- Creation of design patterns and best practices for implementing **Cognition-Oriented Design (COD)** in various domains

- Investigation of the relationship between **Cognition-Oriented Design (COD)** and existing software engineering methodologies
- Development of metrics and evaluation frameworks for measuring the effectiveness of **Cognition-Oriented Design (COD)** implementations
- Exploration of the application of **Cognition-Oriented Design (COD)** to emerging technologies such as edge computing and IoT systems

Note: This PDF paper was written using Cursor and AI, starting from my initial Markdown file available at <https://github.com/flabriola/Cognitive-Oriented-Design>.

References

References

- [1] American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). <https://doi.org/10.1037/0000165-000>
- [2] Hello Monday. (2024). *WOVE - Interactive Design Experience*. <https://www.hellomondays.com/work/wove>
- [3] Model Context Protocol. (2024). *Standardized interaction with large language models*. <https://modelcontextprotocol.io>
- [4] Human-AI Interaction. (2023). *Designing for human-machine collaboration*. ACM Digital Library.
- [5] Intelligent Systems Design. (2024). *Paradigms and methodologies for AI-powered systems*. IEEE Xplore.
- [6] Event-Driven Architecture. (2023). *Principles and patterns for reactive systems*. O'Reilly Media.
- [7] Automation Design. (2024). *Best practices for human-automation interaction*. ACM Digital Library.
- [8] Design Paradigms. (2023). *Evolution of design thinking in software engineering*. Springer.